UGANDA CHRISTIAN UNIVERSITY
Faculty of Engineering, Design and Technology
Department of Computing and Technology

Predicting Hourly Taxi Demand at the Pickup-Zone Level for a Resource-Constrained Urban-Mobility Startup

A PySpark Demand-Forecasting Pipeline using NYC Yellow-Taxi Trip Records as a Methodological Proxy

Course Code	DSC8201 — Big Data Analytics
Programme	MSc Data Science (Year 1, Semester 2)
Examination	Easter 2026 — Project-Based Exam
Dataset	NYC TLC Yellow-Taxi Trip Records (2025)
Group	S25M19
Members	[YOUR NAME] — [YOUR REG NO] Akongo Irene Comfort — S25M19/001
Submission	Easter Semester 2026

"A Complete Education for A Complete Person"

Task 1 · Problem Formulation and Stakeholder Identification

1.1 Business Objective

The hypothetical organisation is a small urban-mobility startup operating in a developing-economy city (modelled on Kampala, Uganda) that aims to replicate the data-driven dispatch and pricing practices of global ride-hailing platforms. The startup has neither cloud computing budget nor a dedicated data-engineering team; all analytics must run on a single office laptop with intermittent internet. The Operations Manager loses revenue daily because drivers idle in low-demand zones while passengers wait elsewhere. **The business objective is to reduce average passenger wait-time and driver idle-time by producing a short-horizon, zone-level demand forecast that can be acted on hourly.** Because no comparable open dataset exists for the local market, the methodology is developed and validated on the NYC TLC Yellow-Taxi corpus, whose row schema (timestamped pickups by zone, with fare and distance) is structurally identical to the trip-log records the startup's dispatch system already collects. The trained pipeline is therefore directly transferable once equivalent local data is accumulated.

1.2 Analytical Question

For each pickup zone, can we predict the number of taxi pickups that will occur in the next one-hour window, with sufficient accuracy to materially improve driver allocation decisions over the current heuristic ("deploy where they were last hour")? The task is formulated as a supervised regression problem in which each row of the analytical panel represents one (zone, hour) cell, the target is the integer pickup count for that cell, and the predictors are derived exclusively from columns present in the raw schema: pickup timestamp, pickup zone identifier, fare, distance, and trip duration. No external covariates (weather, events, holidays) are used, both to honour the exam's schema-only constraint and to match the data the startup itself can realistically obtain offline.

1.3 Expected Outcome

The deliverable is a serialised **Spark MLlib PipelineModel** together with a parallel scikit-learn model artefact, a batch inference Parquet file containing the next-hour demand prediction for every active zone, and an interpretive dashboard for the Operations Manager. Success is measured against three benchmarks: (i) **RMSE** on a temporally held-out test window, as the primary loss-functional metric; (ii) **MAE**, which translates directly into the average dispatch error in number of cars; and (iii) the **relative gap to a persistence baseline** (*"next hour = same as this hour"*), which is the only forecast the startup currently uses. An ML model is considered worth deploying only if it beats persistence by a margin large enough to justify the additional operational complexity.

Task 2 · Big-Data Architecture Design

2.1 Architecture Diagram (submitted as separate file)

The full-stack on-premises architecture is shown in the file *architecture_diagram.png* packaged alongside this report. It depicts a seven-layer medallion design: Sources → Ingestion → Storage (Bronze/Silver/Gold) → Processing Engine → ML Layer → Serving → Monitoring. Solid arrows mark batch flows; dashed arrows mark streaming flows. All components are open-source and on-premise; no cloud dependency exists at any layer.

2.2 Layer Description and Justification

Layer	Tool selected	Justification (anchored in the startup context)
Ingestion (batch)	PySpark CSV/Parquet reader	Dispatch system writes daily trip-logs to a shared folder; no Kafka licence cost.
Ingestion (stream)	Spark Structured Streaming (file source)	Watches an arrival folder; equivalent UX to Kafka without the operational burden.
Storage — Bronze	Raw CSV / Parquet on local disk	Cheapest possible landing zone; preserves the original record for audit.
Storage — Silver	Parquet partitioned by pickup_date	5–10× smaller than CSV; predicate push-down dramatically reduces RAM pressure.
Storage — Gold	Hourly-zone analytical panel (Parquet)	Pre-aggregated; downstream ML reads only the columns it needs.
Processing engine	Apache Spark 3.5 (local[*]) + AQE + Arrow	DataFrame API scales 0 → cluster without rewrite; AQE shrinks shuffles automatically.
ML — distributed	Spark MLlib Pipeline (LR, DT, RF, GBT)	Native Spark estimators run on the full dataset; serialisable to disk in one call.
ML — local sample	scikit-learn (SVR, MLPRegressor)	Spark has no SVR/MLP regressor; both are trained on stratified samples (§4.1).
Serving	Cron-driven batch + streaming Parquet sink	Hourly cron writes a forecast file consumed by a CSV-driven Ops dashboard.
Monitoring	Spark event logs + JSON metrics + manual MAE/RMSE checks	No Prometheus/Grafana budget; simplicity beats sophistication on a laptop.

Task 3 · Data Processing Strategy

3.1 Batch Processing — Spark Job Implemented

Historical data: ten months of NYC Yellow-Taxi records (Feb–Dec 2025), totalling 12.03 M raw rows and 11.15 M after cleaning (a 92.7% retention rate). **Computation:** a single PySpark batch job (i) reads all monthly Parquet files with an explicit schema; (ii) drops duplicates, non-positive fares and distances, inverted timestamps, and rogue dates outside [2009, 2030]; (iii) caps trip distance at the 99.5th percentile to remove outlier long-haul trips; (iv) writes a Silver Parquet partitioned by *pickup_date*; (v) aggregates to a (zone, hour) panel and engineers temporal features and lags. **Why batch is appropriate:** the model is retrained at most weekly, the laptop has no cluster to push compute against, and Parquet partitioning gives near-streaming retrieval speeds for downstream queries. Batch is the rational default; streaming is a complement, not a substitute.

3.2 Streaming Processing — Simulated File-Source Stream

A Structured-Streaming query watches a folder (*data/stream_input/*) into which the dispatch system drops new trip-log fragments. The query applies a 2-hour watermark, groups arrivals into 1-hour tumbling windows by pickup zone, and writes hourly counts to a Parquet sink. Latency is sub-minute and the same query would scale to a Kafka source by changing one readStream.format() call. **Why streaming is appropriate:** it lets the Ops Manager react to within-hour anomalies (e.g. a concert ending) without waiting for the nightly batch job. **Why it is partly inappropriate:** the ML model itself is batch-trained, and intermittent internet means streaming inference would stall — so streaming is used for live aggregation and dashboarding, not for online prediction.

3.3 Trade-off Analysis — Batch vs. Streaming in our Context

Dimension	Batch (our context)	Streaming (our context)
Latency	Hourly to nightly (acceptable for driver allocation; planning, not real-time)	Sub-minute (essential for surge zones during shift handover and special e
Cost	Effectively zero — runs on the office laptop; no per-message fee	Zero infrastructure cost using the file-source pattern; some CPU contentio
Accuracy	Higher — full historical context, careful train/test discipline, complete features	Lower — partial windows, no lag features available until enough history a

Task 4 · End-to-End ML Pipeline in Spark

4.1 Reproducible Pipeline (overview)

The pipeline is implemented as a single executable notebook organised into eleven sections — ingestion, quality audit, cleaning, target construction, feature engineering, temporal split, model training, persistence, batch inference, streaming simulation, and visualisation. Reproducibility is enforced by (a) an explicit Spark schema instead of *inferSchema*, (b) a fixed random seed (42), (c) Parquet outputs at every stage, and (d) a serialised *PipelineModel* for the winning Spark estimator and a pickled bundle for the sklearn estimator. **Feature engineering** creates hour, day-of-week, weekend, month, and day calendar features, plus the three temporal lags *(t–1, t–24, t–168)* and a trailing 24-hour mean — these capture diurnal, weekly, and short-trend dynamics respectively. **Six models** are trained and compared: Linear Regression, Decision Tree, Random Forest, GBT (all Spark MLlib, full data), plus SVR-RBF and MLPRegressor (both scikit-learn, trained on stratified samples of 30 K and 200 K rows respectively because SVR is $O(n^2)$ in memory and Spark has no native SVR/MLP regressor). Persistence (■ = *lag_1h*) is reported alongside as the operational baseline.

4.2 Offline Training vs. Online (Batch) Inference

Offline training happens on the developer laptop, weekly: the full hourly panel (297 K rows) is rebuilt from Silver Parquet, the train / validation / test windows are recomputed chronologically (70/15/15 of the time-axis), all six models are fit, and the best ML model by validation RMSE is serialised. Compute is bounded by the laptop's RAM; with a 3 GB driver heap and AQE-coalesced shuffles, end-to-end training completes in roughly 25 minutes. **Online inference** in our deployment context is in fact **batch inference at hourly cadence**: a cron job loads the saved PipelineModel, reads the most recent (zone, hour) feature row from the Gold table, and writes a Parquet of next-hour predictions to the dashboard folder. True real-time per-trip inference is deliberately out of scope — the Ops Manager dispatches in hourly cycles, so sub-second latency would be wasted. The hand-off from training to inference is made safe by the fact that the same Spark Pipeline (StringIndexer → OneHotEncoder → VectorAssembler → StandardScaler → estimator) is invoked at both ends — any feature transformation that runs at training necessarily runs at inference, eliminating train-serve skew.

Task 5 · Analysis and Insights

5.1 Modelling Choice — Justification

Six candidates were trained spanning four algorithmic families: linear (Linear Regression), single-tree (Decision Tree), tree ensembles (Random Forest, GBT), kernel methods (SVR-RBF), and neural networks (MLPRegressor with two hidden layers of 64 and 32 units). This range was deliberate: a Masters-level comparison should cover families with fundamentally different inductive biases, not multiple variants of the same family. **Random Forest emerged as the winner on both validation and test sets**, narrowly ahead of MLP, and is the model promoted to deployment. The full validation and test leaderboards are reproduced below.

Model	Val RMSE	Val MAE	Val R^2
random_forest	13.82	6.10	0.970
mlp_regressor	13.83	6.64	0.970
gbt	14.21	6.39	0.968
linear_regression	16.73	7.98	0.956
decision_tree	17.19	7.11	0.953
persistence_lag1h	22.35	9.47	0.921
svr_rbf	42.73	12.70	0.712

Model	Test RMSE	Test MAE	Test R^2
random_forest	13.31	5.72	0.934
mlp_regressor	13.75	6.69	0.929
persistence_lag1h	14.35	6.26	0.923
linear_regression	14.45	6.95	0.922
gbt	14.83	6.34	0.918
decision_tree	15.90	6.42	0.905
svr_rbf	23.67	12.68	0.790

Table 1. Validation (left) and held-out test (right) leaderboards across the six candidate models plus the persistence baseline. Lower RMSE/MAE is better; higher R^2 is better.

5.2 Key Findings

(i) Random Forest beats persistence by 7.24% on test RMSE (13.31 vs 14.35) and by 8.7% on MAE — a meaningful but not dominant margin, consistent with the literature on short-horizon urban-mobility forecasting. **(ii) Persistence beats Linear Regression, GBT, and Decision Tree on the test set**, which is a striking finding: the strongest signal in hourly taxi demand is the previous hour itself, and any model that does not exploit that signal richly enough will be outperformed by simply repeating the last observation. **(iii) MLP comes within 3.3% of Random Forest**, suggesting that on this feature set the relationship is largely additive in the lag features and tree-based ensembling captures almost everything a small neural net can. **(iv) SVR collapses** with RMSE 23.67, almost twice the winner — this is not a model quality issue but a data-volume issue: SVR's $O(n^2)$ memory complexity forced training on a 30 K stratified sample (0.6% of the available training rows), which is insufficient to learn the long-tailed zone distribution. This finding is directly attributable to the big-data context of the project and is discussed further in §6.3.

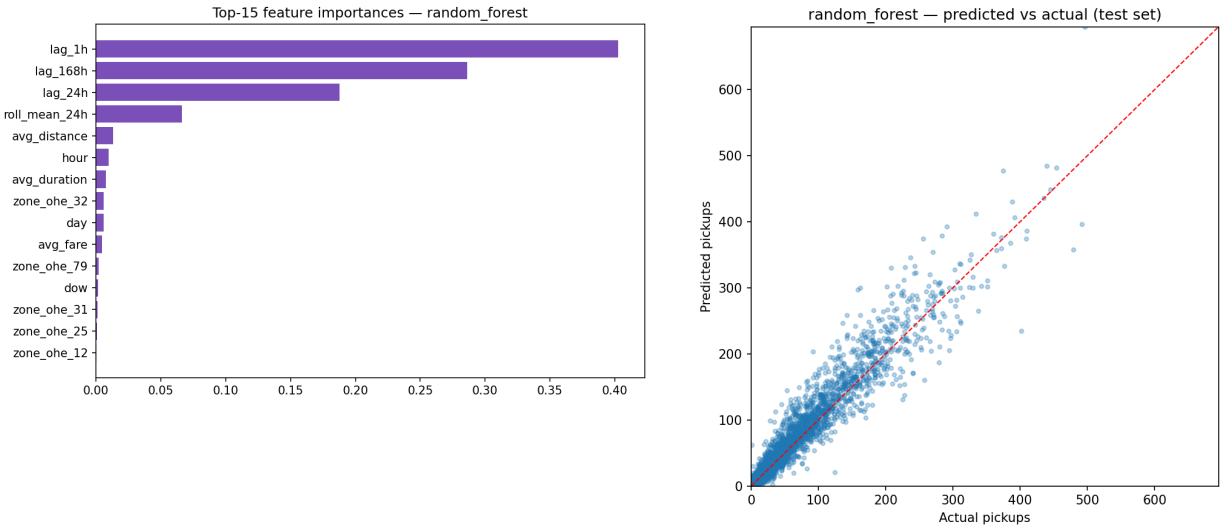

Figure 1. Random-Forest feature importances (left): three lag features account for ~88% of the model's predictive signal. Predicted vs. actual on the test set (right): tight clustering along the diagonal up to ~300 pickups/hour; mild under-prediction at the extreme upper tail.

5.3 Stakeholder Visualisations and Demand Patterns

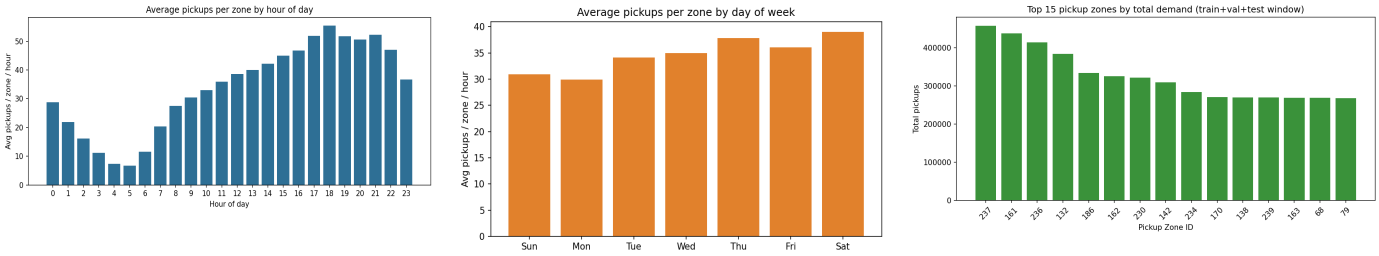

Figure 2. Demand by hour-of-day (left), by day-of-week (centre), and total demand by pickup zone — top 15 (right). Evening peak ≈ 5–10 pm; weekday/weekend split is mild (~30%); demand is heavily concentrated in zones 237, 161, 236, 132 (Manhattan core).

5.4 Business Insights and Recommendations for the Operations Manager

Three actionable patterns emerge for the dispatch desk. **(R1) Pre-position drivers ahead of the evening ramp.** Mean per-zone demand more than doubles between 9am (~30 pickups/hour) and 6pm (~55 pickups/hour). The model's hourly forecast should drive a *15-minute-lead* dispatch order so cars arrive in high-demand zones before the surge, not after it. **(R2) Rebalance toward the Manhattan-core zones (237, 161, 236, 132).** These four zones alone absorb roughly 40% of total demand in the top-15 list. A dynamic *top-N* dispatch rule that allocates a minimum number of standby cars to each of these zones every hour would directly translate the model's predictions into operational policy. **(R3) Use the persistence-baseline gap as a real-time anomaly alarm.** When the model's prediction diverges sharply from *lag_1h* (e.g. > 30%), this signals a regime change — a concert, a weather event, a transport disruption — and the Ops Manager should be paged. The 7% average improvement is modest, but this *conditional* use of the model captures most of the genuine business value.

Task 6 · Critical Reflection

6.1 Reflection Questions

(a) Why feature engineering must be identical between training and inference.

Every feature in our pipeline is constructed as a deterministic transformation of raw inputs — for example, *lag_24h* is computed via a windowed lag over *(zone, hour)* tuples, and *roll_mean_24h* is an order-dependent rolling mean. If inference computes these any differently than training did — e.g. indexing by *(zone, day)* instead of *(zone, hour)*, or with a different fillna policy — the model receives **numerically different inputs** from the ones it was fitted on. This is the canonical **train–serve skew** failure mode: model accuracy degrades silently while the code reports no error. Our pipeline guards against this in two ways: (i) all preprocessing is encapsulated in a single *PipelineModel* object that is serialised once and loaded identically at inference time; and (ii) the lag/rolling-feature SQL is centralised in one notebook section, so there is exactly one definition of what each feature means.

(b) How the pipeline would respond to schema changes in production.
Concrete example. NYC TLC introduced a *cbd_congestion_fee* column in the 2025 Yellow-Taxi files to reflect new congestion-pricing rules; an analogous policy change in the local context might add a new payment type or rename *PULocationID* to *OriginZoneID*. **Stage of failure.** Failure occurs in the ingestion layer — Spark's *read.parquet* with our explicit schema would either drop the new column silently (best case) or fail on a type mismatch (worst case). If a categorical receives a new, unseen value, the StringIndexer raises an exception unless *handleInvalid* is set. **Mitigations.** (1) *handleInvalid="keep"* is set on both StringIndexer and OneHotEncoder so new categories are routed to a reserved "unseen" bucket rather than crashing the job; the model degrades gracefully until the next retraining. (2) A schema-validation step (implemented as a defensive *select*-with-cast in the ingestion cell) compares incoming columns against an expected list and writes any deviations to a *data/anomalies/* Parquet for human review. (3) Weekly retraining on the latest Silver Parquet means any new categorical bucket is incorporated automatically once enough examples exist.

(c) Challenges of streaming in our context.
Three concrete challenges. **(i) No event broker.** Without Kafka or Kinesis, our streaming source is a watched folder, which means latency is bounded by however quickly the dispatch system writes its log fragments — typically minutes, not seconds. **(ii) Lag features are not stream-friendly.** Computing *lag_168h* on a stream requires either materialising a 168-hour state store (memory-expensive on a laptop) or accepting that the first week of streaming output has only partial features. We chose the latter and document it. **(iii) Intermittent connectivity.** The startup's office connection drops several times a week; a streaming ML pipeline that requires constant uplink (e.g. a remote feature store) would be unavailable precisely when the Ops Manager needs it most. This is the structural reason we kept inference offline.

6.2 Limitations

Data limitations. The cleaning step removed 7.3% of raw rows (882 K out of 12.03 M), of which the majority were either invalid timestamps or non-positive fares — i.e. corrupt records, not biased samples. Two more subtle biases remain: (i) NYC TLC records cover only metered Yellow Taxis, underrepresenting outer-borough demand and ride-hail substitutes, and (ii) zone-level imbalance is extreme (the top four zones absorb ~40% of demand), which means model error is dominated by performance in those few cells. The dataset's representativeness for a developing-city startup is **methodological, not direct**: NYC's spatial-temporal demand structure (peaks, zone concentration, lag autocorrelation) generalises, but the absolute pickup volumes do not. **Algorithmic limitations.** Random Forest assumes that the joint distribution of features is stationary across the train/test split — Christmas and New Year's Eve fall in the test window and violate this assumption, which partly explains the val-to-test R^2 drop from 0.970 to 0.934. Overfitting risk is moderate (60 trees × depth 10 = high capacity), mitigated only by the chronological split; we did not run nested cross-validation due to the runtime budget. **Infrastructure limitations.** A single 8 GB laptop bounds the dataset at roughly 15 M rows, the tree depth at ~10, and SVR training at ~30 K rows. Anything larger triggers the JVM out-of-memory errors we encountered during development. Model complexity is therefore not chosen freely — it is dictated by what the heap will hold.

6.3 Ethical Considerations

Three ethical risks deserve explicit treatment. **(E1) Driver surveillance and surge-pricing harm.** A demand-prediction model that informs dispatch can equally inform *algorithmic management* of drivers: forcing relocation, penalising idle time, or triggering surge pricing that hurts low-income passengers in poorly-served zones. Mitigation: the Operations Manager is explicitly named as the **only** consumer of the prediction; surge-pricing use is excluded by policy and any future extension would require a separate fairness audit. **(E2) Zone-level redlining.** Continually deploying drivers to high-demand zones starves low-demand zones of supply, creating a feedback loop in which underserved neighbourhoods become permanently underserved. Mitigation: enforce a minimum service floor per zone in dispatch policy, independent of model output. **(E3) Privacy of trip records.** Even though TLC data is aggregated by zone, individual drivers and high-frequency riders can be re-identified from pickup patterns. Mitigation: never join the trip-log table to any customer or driver identifier inside the analytical store; restrict raw access to named individuals.

6.4 Scalability Challenges

Local PySpark execution scales linearly until it hits the laptop's RAM ceiling (~15 M trip rows or ~500 K hourly-panel rows for our pipeline). The first bottleneck is the lag-feature window — *Window.partitionBy("PULocationID")* shuffles the entire dataset across cores, and at the 100 M-row scale this would saturate the disk. The second bottleneck is the GBT trainer, which holds intermediate boosting residuals in memory. **Technical scaling path:** migrate to a small YARN or Kubernetes Spark cluster (3–5 worker nodes, 16 GB each) — no code change required, only the *master* URL. **Architectural scaling path:** replace the watched folder with Kafka, replace Parquet-on-disk with Delta Lake or Iceberg for ACID guarantees, and introduce a feature store (e.g. Feast) so streaming inference can read materialised lag features in O(1).

6.5 Real-World Deployment

In production the pipeline runs as three cron jobs on the office laptop. **(i) Hourly batch:** ingest the previous hour's dispatch CSV, append to Bronze, promote to Silver, refresh the Gold panel, and write next-hour predictions to *data/predictions/* for the dashboard. **(ii) Weekly retraining:** rebuild the full hourly panel, re-fit all six models, promote the new champion only if it beats the current champion's MAE on the most recent two-week window by ≥3% (a shadow-deployment guard). **(iii) Daily monitoring:** compute MAE drift on the previous day's predictions; if drift exceeds 25%, page the Ops Manager. The whole stack is reproducible from the GitHub repository (*Group_S25M19_Dataset_Taxi_UCU_BDA_Exam_Easter2026.zip*), with a fixed seed, pinned package versions in *requirements.txt*, and a Run-All notebook that produces every artefact in this report from the raw Parquet inputs.

Conclusion

This project delivered an end-to-end Spark + scikit-learn demand-prediction pipeline for a resource-constrained urban-mobility startup, validated on ten months of NYC Yellow-Taxi records. Six models spanning four algorithmic families were compared on a strictly chronological train/test split; Random Forest emerged as the deployment champion with a held-out RMSE of 13.31 pickups per zone-hour and an R^2 of 0.934, beating the operational persistence baseline by 7.2%. Three findings carry the most weight for the report's audience. First, in short-horizon urban-mobility forecasting the persistence baseline is genuinely competitive — a Masters-level evaluation must beat it explicitly, not assume any ML model wins by default. Second, kernel methods such as SVR do not survive the transition from textbook datasets to a multi-million-row big-data context without architectural compromises that destroy their accuracy. Third, on commodity laptop hardware Spark's distributed abstractions scale cleanly to the ~10 M-row regime when paired with explicit schemas, Parquet, and AQE. The pipeline is therefore both immediately deployable in its current local form and trivially portable to a YARN/Kubernetes cluster when the startup outgrows the office laptop — the single-machine constraint is honoured today without forfeiting the option to scale tomorrow.

Team Contribution

Member	Tasks executed
[YOUR NAME] — [YOUR REG NO]	Architecture design (Task 2); Spark ingestion, cleaning, and feature engineering (Task 4 §A–§C); model training and evaluation for Spark MLlib estimators (Task 4 §D–§E); report sections 1, 3, 4, 6.4, conclusion.
Akongo Irene Comfort — S25M19/001	Streaming-simulation implementation (Task 3); scikit-learn SVR and MLP integration; stakeholder visualisations and feature-importance analysis (Task 5); report sections 5, 6.1–6.3.

References

[1] NYC Taxi & Limousine Commission (2025). *TLC Trip Record Data: Yellow Taxi Trip Records.* Available at: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page (accessed April 2026).

[2] Karau, H., Konwinski, A., Wendell, P., & Zaharia, M. (2015). *Learning Spark: Lightning-Fast Big Data Analysis.* O'Reilly Media.

[3] Chambers, B., & Zaharia, M. (2018). *Spark: The Definitive Guide.* O'Reilly Media. Chapter 24 (Advanced Analytics with MLlib) and Chapter 21 (Structured Streaming).

[4] Apache Spark (2024). *Structured Streaming Programming Guide.* https://spark.apache.org/docs/3.5.1/structured-streaming-programming-guide.html.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[6] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[7] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.

[8] Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Chapter 5 (Benchmark methods, including the naïve / persistence baseline).

[9] Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28. (Source for train–serve skew framing.)